

Figure 6: Output of the dial tone example in GNU Radio Companion

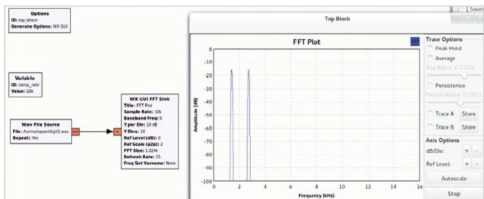


Figure 7: FFT plotting

3. Configure, compile and install GNU Radio.

Interested readers can go to the link given in Reference 6, where all the steps for the complete manual installation are listed. If this approach is used, then several commands should be executed with care and the many dependencies should also be installed carefully.

Demo example 1 [dial tone generator]

Once GNU Radio is completely installed, it offers certain sample code examples. One such example is to generate a dial tone using two sine waves of different frequencies. Figure 5 presents the GNU Radio Companion interface to enable the dial tone implementation. Appropriate connections between various blocks generate a flow graph structure, and once the flow graph is executed, it generates the corresponding Python code in the same directory.

Executing either the generated Python code from the terminal or the flow graph directly from GNU Radio Companion will produce the output shown in Figure 6.

Demo example 2 [FFT representation]

Figure 7 shows a simple flow graph, which accepts a signal and displays its frequency domain representation through Fast Fourier Transform (FFT).

Signal processing block

There are basically three types of signal processing blocks in GNU Radio Companion. These are listed below.

1. **Source blocks:** These act as input providers and have only outgoing connections.
2. **Sink blocks:** These act as output seekers, and have only incoming connections or links.
3. **Processing blocks:** These perform some processing (such as modulation, filtering, etc), and can have both incoming and outgoing links.

Hence, every flow graph is characterised by minimum one source block, one sink block and some processing blocks.

Applications

We have discussed only small examples as a beginner's guide to readers, but one can also implement highly complex applications using GNU

Radio. Typical applications include AM receiver, FM receiver, intelligent spectrum sensing, digital radio astronomy, and many more. 

References

- [1] Sapan H. Mankad and S.N. Pradhan. Application of Software Defined Radio for Noise Reduction using Empirical Mode Decomposition. In *Advances in Computer Science, Engineering & Applications*, pages 113–121. Springer Berlin Heidelberg, 2012
- [2] Bard, J., Kovarik V.J. Software Defined Radio - The Software Communications Architecture. John Wiley and Sons (2007)
- [3] Kuntal Ray. An Introduction to GNU Radio
- [4] Ivan Seskar. Hands-on-Tutorial, Future Internet Summer School 2009 Part 2. The State University of New Jersey
- [5] <https://gnuradio.org/redmine/users/3733>
- [6] Installing GNU Radio on Ubuntu <https://gnuradio.org/redmine/projects/gnuradio/wiki/UbuntuInstall>

By: Sapan H. Mankad

The author works as assistant professor in the CSE Department of Institute of Technology, Nirma University. His research interests include speech signal processing, machine learning and ICT tools in education. He can be contacted at sapanmankad@nirmauni.ac.in. He blogs at <http://www.shmive.wordpress.com> and his website is <https://sites.google.com/a/nirmauni.ac.in/sapan-mankad/home>